Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #3.1

Signal Acquisition. Binary signals.

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents

1 Objective and Technical Analysis **2**
1.1 Digital Sensor Interfaces . . . 2
1.2 Binary Signal Conditioning . . . 3
1.2.1 Threshold Detection with Hysteresis . . . 3
1.2.2 Software Debouncing via Counter Method . . . 4

2 System Design **4**
2.1 System Architecture . . . 4
2.2 Flowchart of the Program . . . 5
2.2.1 Individual Task Flowcharts . . . 5
2.2.2 General System Flowchart . . . 8
2.2.3 Threshold Alert State Machine . . . 9

3 Implementation and Results **10**
3.1 Software Architecture . . . 10
3.2 Task-Specific Implementation Details . . . 11
3.2.1 Acquisition Task . . . 11
3.2.2 Threshold Task . . . 11
3.2.3 Report Task . . . 11
3.3 Layered Design (Hardware-Software) . . . 12
3.3.1 Distance Sensor Driver . . . 12
3.4 Electrical Schematics . . . 13
3.5 Simulation . . . 14
3.6 Challenges . . . 14

4 Conclusion **15**

References **16**

A Source Code **17**

1 Objective and Technical Analysis

The objective of this laboratory work is to study signal acquisition and conditioning techniques for embedded systems by implementing a real-time distance monitoring system with threshold-based binary alerting. The system uses an HC-SR04 ultrasonic distance sensor connected to an ATmega328P microcontroller running FreeRTOS, implementing periodic sensor acquisition, binary signal conditioning through hysteresis and debouncing, and structured reporting via UART.

Through this work, the following learning outcomes are pursued:

- Understanding sensor acquisition principles for digital sensors and the signal conditioning pipeline.
- Implementing threshold-based binary signal detection with hysteresis to prevent oscillation near boundary values.
- Applying software debouncing techniques (counter-based) to validate the persistence of state transitions.
- Designing a modular multitask architecture for sensor acquisition, conditioning, and reporting using FreeRTOS.
- Utilizing the CRTP (Curiously Recurring Template Pattern) for compile-time polymorphism in embedded C++.

1.1 Digital Sensor Interfaces

Digital sensors provide data directly in digital format, eliminating the need for ADC conversion. They communicate with the microcontroller through various digital protocols such as I2C, SPI, UART, 1-Wire, etc. Each protocol has its own advantages and limitations in terms of speed, complexity, and number of pins used.

Digital interface sensors are generally more complex devices, providing data directly in digital format through a standardized interface. These sensors frequently include built-in conditioning circuits, either in a dedicated integrated circuit (IC) or implemented on a PCB that incorporates both the sensitive element and signal conditioning functions.

The main types of digital interfaces used for sensors include:

- **I2C (Inter-Integrated Circuit)**: Uses only two lines (SDA – data and SCL – clock), allows multiple device connection on the same bus with individual addressing, and is ideal for sensors that do not require rapid data transfer.
- **SPI (Serial Peripheral Interface)**: Provides full-duplex communication via four lines (MOSI, MISO, SCK, SS), allows higher transfer speeds compared to I2C, and requires a separate selection line for each sensor.

- **UART (Universal Asynchronous Receiver/Transmitter)**: Uses two lines (TX – transmit and RX – receive) for bidirectional asynchronous communication, frequently used for simple communication between two devices.
- **1-Wire**: A digital protocol that uses a single line for bidirectional communication, suitable for sensor networks where space and wiring complexity are limiting factors.

The HC-SR04 ultrasonic distance sensor used in this laboratory work uses a simple digital pulse-based interface: a trigger pulse is sent on the TRIG pin, and the sensor responds with a pulse on the ECHO pin whose width is proportional to the measured distance. This is neither a standard bus protocol nor an analog signal, but a time-of-flight digital measurement that can be read using the `pulseIn()` function on Arduino-compatible platforms.

1.2 Binary Signal Conditioning

Binary signal conditioning transforms a continuous (or discrete numerical) sensor value into a binary decision: alert or normal. This is essential in monitoring systems where the goal is to detect whether a measured quantity exceeds or falls below certain thresholds.

1.2.1 Threshold Detection with Hysteresis

A simple threshold comparator triggers an alert when the measured value crosses a single boundary. However, when the signal fluctuates near the threshold, this causes rapid alternation between alert and normal states — a phenomenon known as "chattering" or "bouncing."

To prevent this, *hysteresis* introduces two separate thresholds: a high threshold T_H and a low threshold T_L, centered around the desired value T_C with a band $\pm h$:

$$T_H = T_C + h, \quad T_L = T_C - h \tag{1}$$

The alert is triggered only when the value exceeds T_H, and cleared only when it drops below T_L (or vice versa for low alerts). This creates a "dead zone" that filters out small oscillations.

The concept is illustrated in Figure 1.

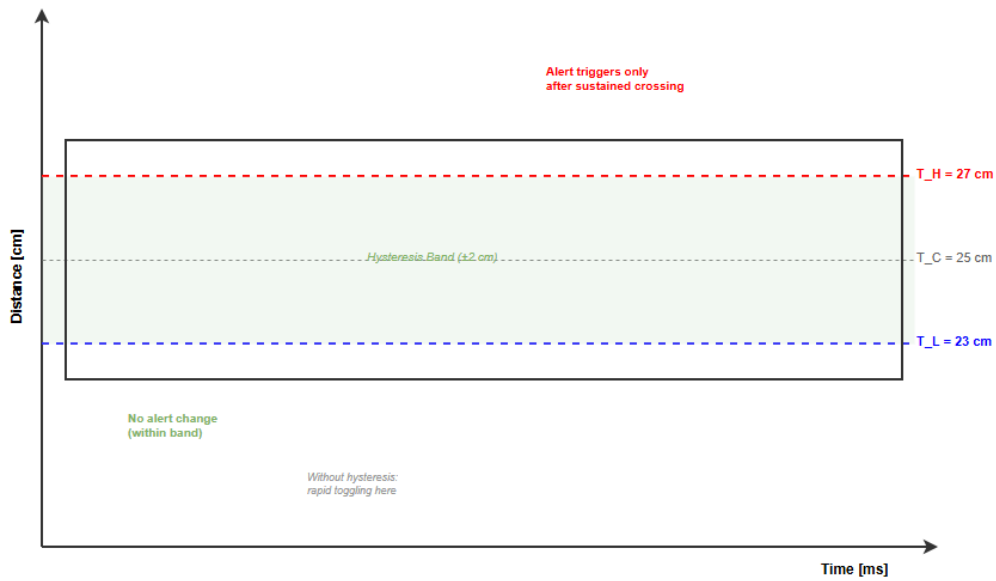

Figure 1: Threshold detection with hysteresis: two thresholds prevent oscillation

1.2.2 Software Debouncing via Counter Method

Even with hysteresis, transient sensor spikes could trigger false alerts. Software debouncing strengthens the decision by requiring multiple consecutive readings in the same state before confirming a transition.

The counter-based debounce algorithm works as follows:

- A counter increments each time the condition is met (e.g., value above T_H).
- If the condition is no longer met before the counter reaches the required threshold, the counter resets.
- Only when the counter reaches a configurable `debounceRequired` value is the state transition confirmed.

This ensures that brief spikes or noise do not trigger state changes, while legitimate sustained threshold crossings are detected after a configurable delay of debounceRequired$\times$ thresholdIntervalMs milliseconds.

2 System Design

2.1 System Architecture

Figure 2 illustrates the overall architecture of the signal acquisition and binary conditioning system. The core components include:

- **Microcontroller (ATmega328P)**: The central processing unit running FreeRTOS, managing three concurrent tasks for acquisition, threshold detection, and reporting.
- **HC-SR04 Ultrasonic Sensor**: Connected via TRIG (pin D9) and ECHO (pin D10) pins, measures distance using time-of-flight of ultrasonic pulses.
- **LEDs**: Two LEDs (Red on D12, Green on D13) provide visual alert feedback. Green indicates normal state, Red indicates active alert.
- **UART Interface**: Used for serial communication at 9600 baud to output periodic structured reports via `printf()`.
- **FreeRTOS Kernel**: Manages preemptive scheduling, task creation, and timing via `vTaskDelay()`.

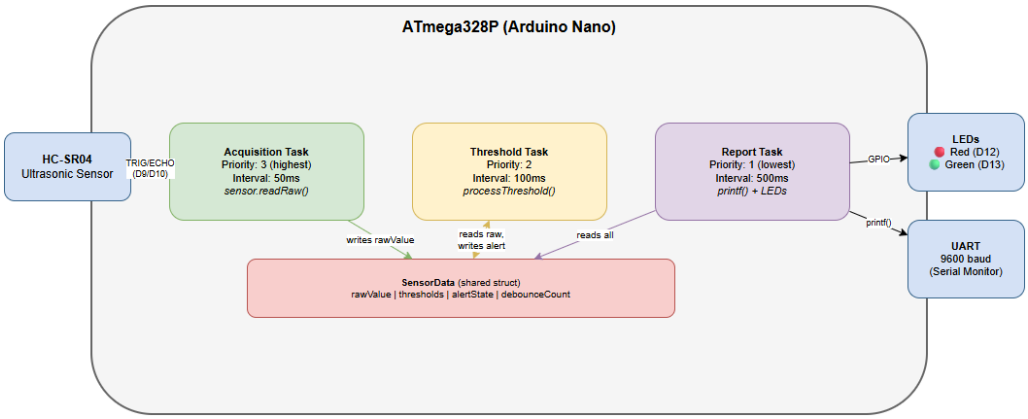

Figure 2: System Architecture Diagram

2.2 Flowchart of the Program

2.2.1 Individual Task Flowcharts

Figure 3 depicts the flow of the Acquisition Task (priority 3, highest). This task is responsible for periodically reading the raw sensor value from the HC-SR04 distance sensor. It calls `sensor.readRaw()` which triggers an ultrasonic pulse and measures the echo time to compute distance in centimeters. The raw value is stored in the shared `SensorData` struct, accessible by the Threshold Task. The task delays for the configured `readingIntervalMs` (50ms) using `vTaskDelay()`.

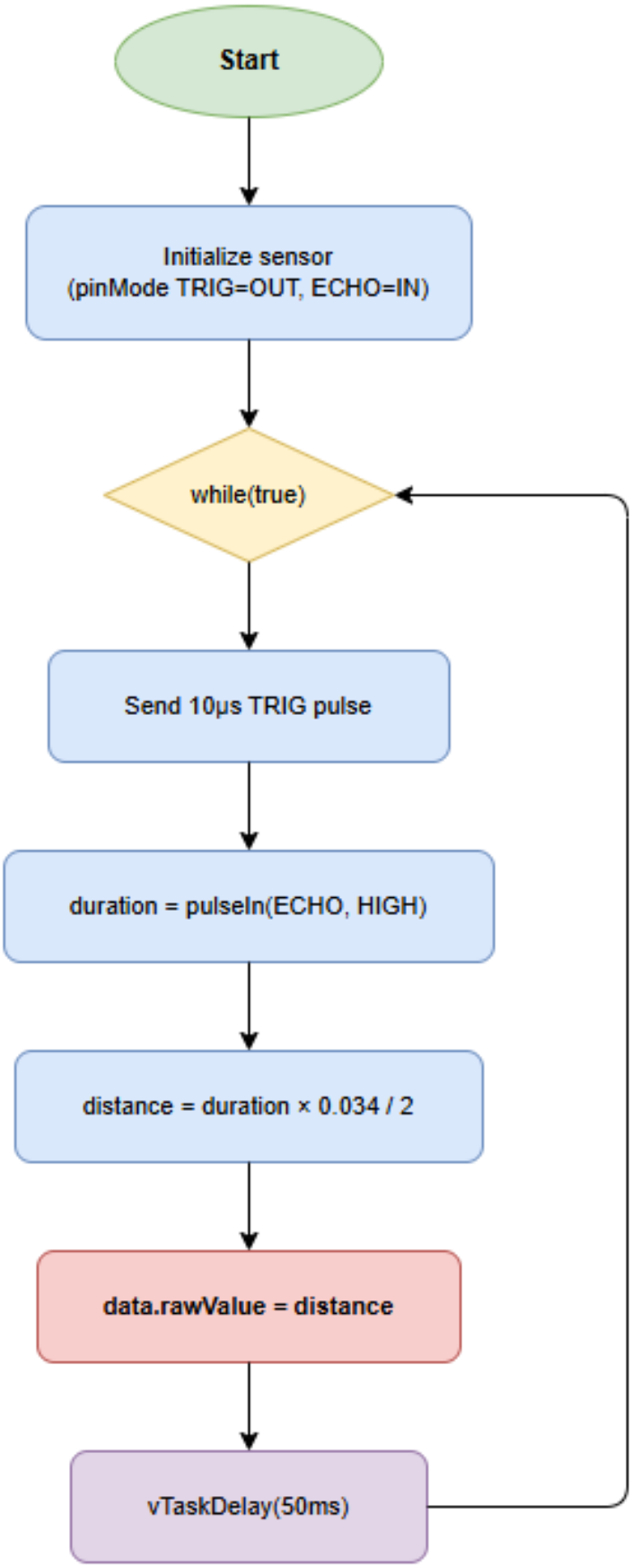

Figure 3: Acquisition Task Flowchart

Figure 4 illustrates the flow of the Threshold Task (priority 2). This task implements a finite state machine with 7 states (see Section 2.2.3) that processes the raw sensor value against configurable thresholds with hysteresis. Each iteration reads the current `rawValue` from the shared data structure, evaluates the current state, and potentially transitions to a new state. The debounce counter ensures state changes are only confirmed after `debounceRequired` consecutive confirmations.

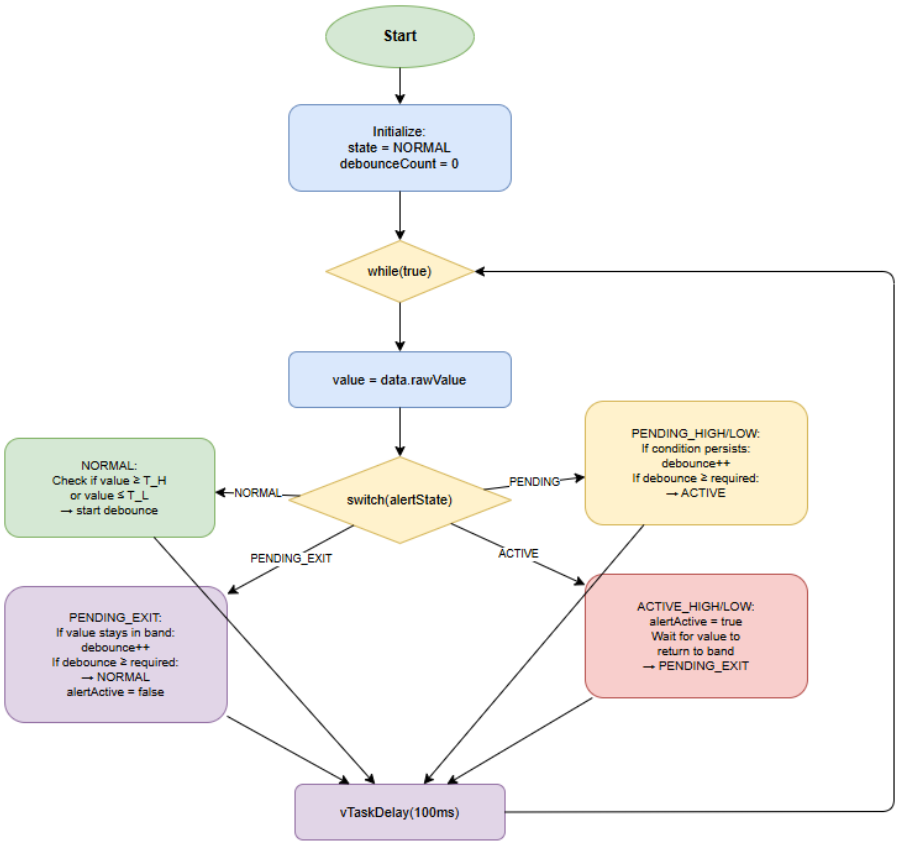

Figure 4: Threshold Task Flowchart

Figure 5 shows the flow of the Report Task (priority 1, lowest). Every 500ms it iterates through all registered sensors, reads their current state, formats a structured report with raw values, threshold bounds, alert state, and debounce count, and outputs it via UART. It also invokes the `onAlertUpdate()` callback to control the LED indicators: Red LED on if any sensor is in alert, Green LED on otherwise.

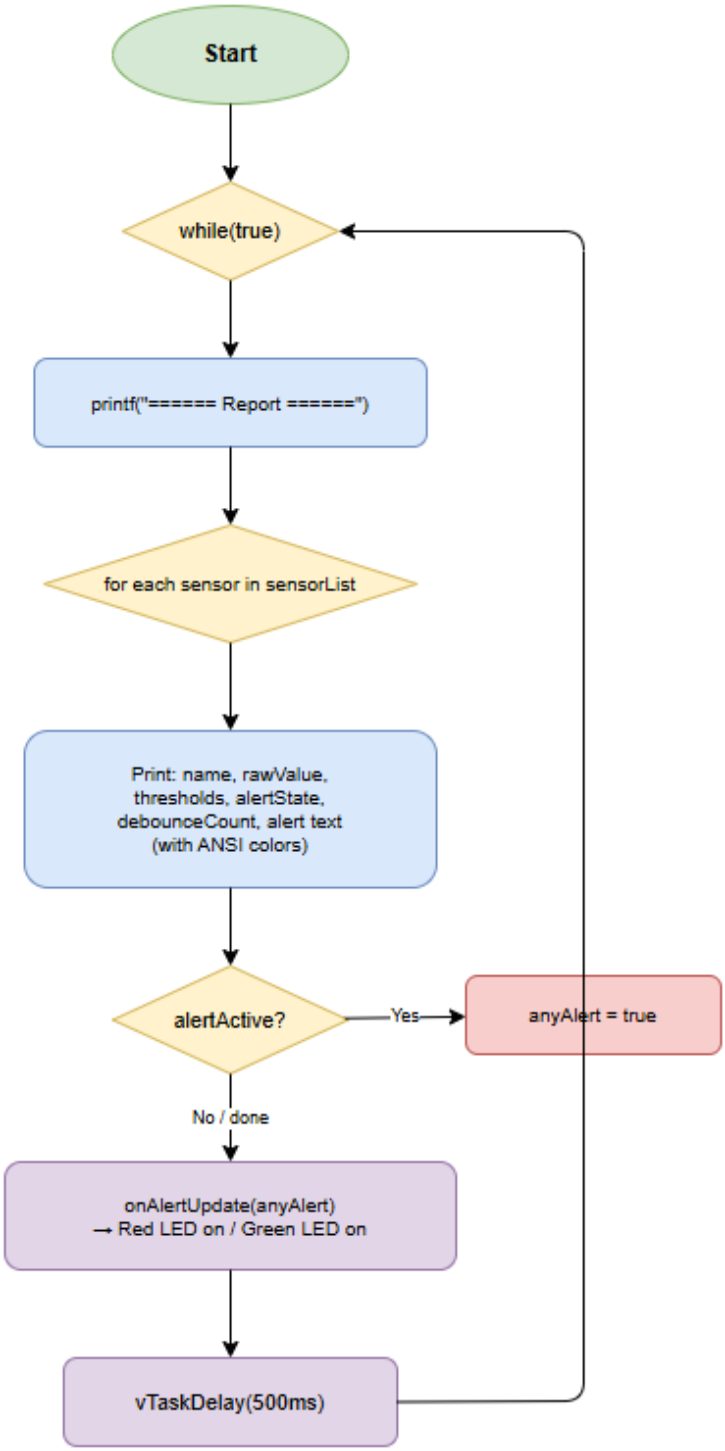

Figure 5: Report Task Flowchart

2.2.2 General System Flowchart

Figure 6 provides an overview of the entire system's flow, showing how the three FreeRTOS tasks interact through the shared `SensorData` structure. The Acquisition Task writes raw sensor values, the Threshold Task reads them and updates alert states, and the Report Task reads all fields for display and LED control. The FreeRTOS kernel

handles preemptive scheduling based on task priorities.

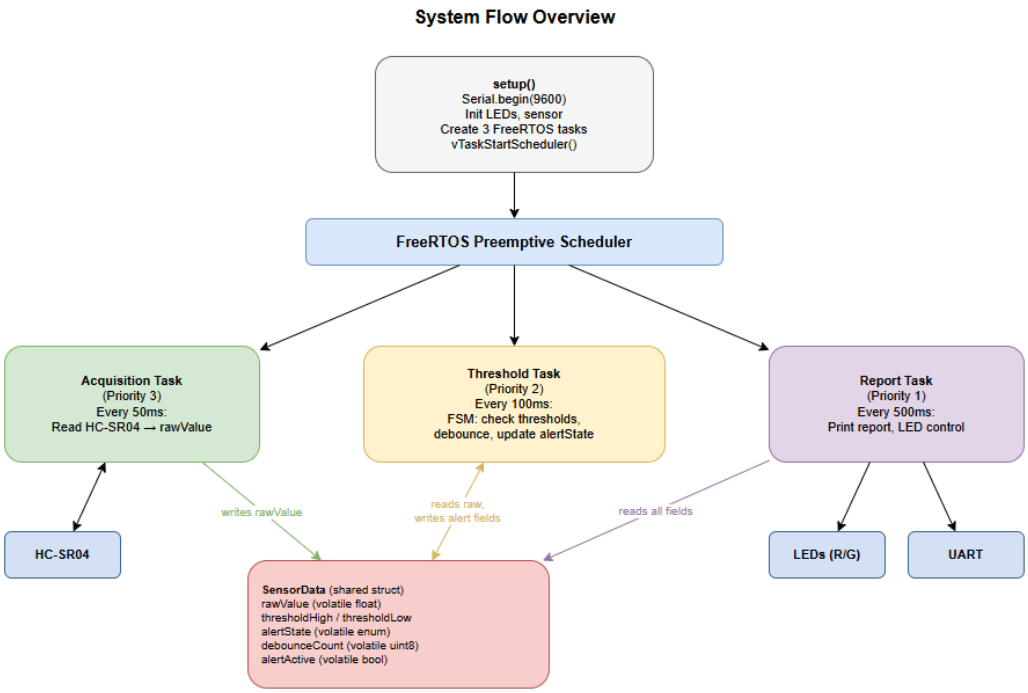

Figure 6: General System Flowchart

2.2.3 Threshold Alert State Machine

The Threshold Task implements a 7-state finite state machine (FSM) to manage alert transitions with hysteresis and debouncing. Figure 7 illustrates the complete state diagram.

The states are:

1. **ALERT_NORMAL**: Value is within the hysteresis band $[T_L, T_H]$. No alert is active.
2. **ALERT_PENDING_HIGH**: Value exceeded T_H, debounce counter is incrementing.
3. **ALERT_ACTIVE_HIGH**: Debounce confirmed, high alert is active.
4. **ALERT_PENDING_EXIT_HIGH**: Value dropped below T_H, debouncing to exit high alert.
5. **ALERT_PENDING_LOW**: Value fell below T_L, debounce counter is incrementing.
6. **ALERT_ACTIVE_LOW**: Debounce confirmed, low alert is active.

7. **ALERT_PENDING_EXIT_LOW**: Value rose above T_L, debouncing to exit low alert.

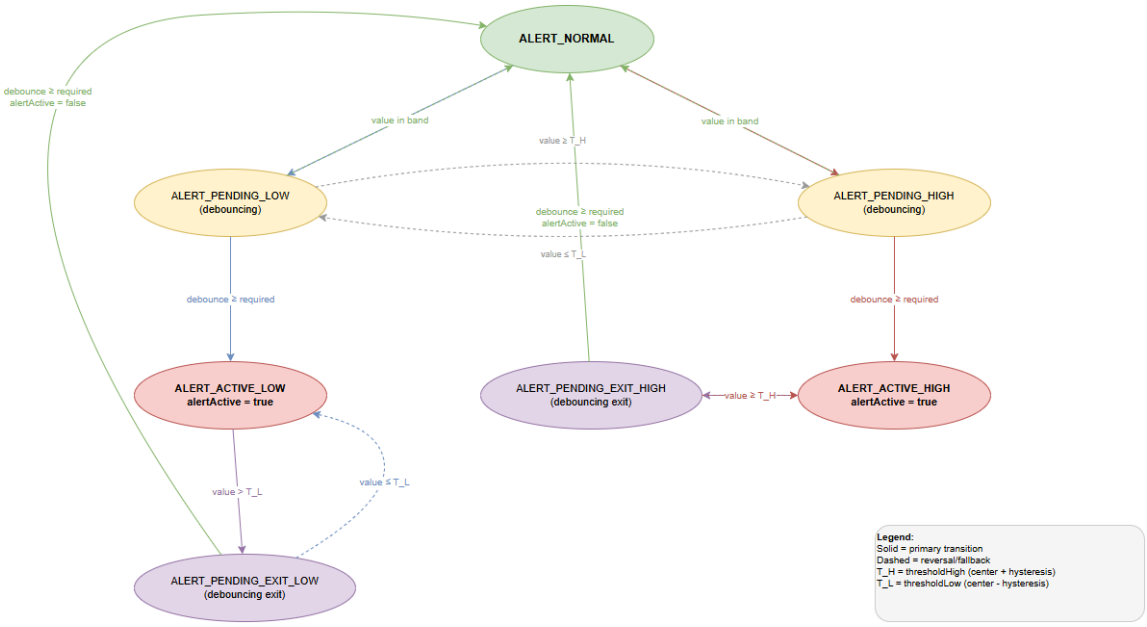

Figure 7: Alert State Machine: 7-state FSM with hysteresis and debouncing

3 Implementation and Results

3.1 Software Architecture

The code is organized into three FreeRTOS tasks, each implemented in separate source files. The shared `SensorData` struct serves as the inter-task communication channel, with the Acquisition Task writing raw values and the Threshold Task updating alert states. The Report Task reads all fields for display purposes.

Key design patterns used:

- **CRTP (Curiously Recurring Template Pattern)**: Both `GenericTask<Derived>` and `Sensor<Derived>` use CRTP to achieve compile-time polymorphism without virtual function overhead — critical for the ATmega328P's limited resources.

- **Template-based Sensor Task**: `GenericReadingSensorTask<SensorType>` is parameterized on the sensor type, allowing reuse with any sensor that provides a `readRaw()` method.

- **Configurable Parameters**: The `SensorConfig` struct encapsulates all tunable parameters (intervals, thresholds, hysteresis band, debounce count, label) in a single configuration object.

3.2 Task-Specific Implementation Details

3.2.1 Acquisition Task

The `GenericReadingSensorTask<DistanceSensor>` template instantiation creates a task that calls `distanceSensor.readRaw()` every 50ms. The `readRaw()` method sends a 10μs trigger pulse, measures the echo duration using `pulseIn()`, and converts to centimeters via the formula:

$$d = \frac{t \times 0.034}{2} \text{ [cm]} \tag{2}$$

where t is the round-trip echo time in microseconds and 0.034 cm/μs is the speed of sound.

3.2.2 Threshold Task

The `ThresholdTask` implements the 7-state FSM described in Section 2.2.3. Key configuration for the distance sensor:

- Center threshold: 25.0 cm
- Hysteresis: $\pm$2.0 cm (high = 27.0 cm, low = 23.0 cm)
- Debounce required: 3 consecutive readings
- Evaluation interval: 100ms

The state machine is implemented as a `switch` statement in `processThreshold()`, with each case handling the transitions for its state. The `volatile` qualifier on shared fields ensures correct access across tasks.

3.2.3 Report Task

The `ReportTask` namespace provides a `run()` function that iterates through a sensor list every 500ms. It uses `splitFloat()` for integer-based float formatting (avoiding expensive `printf` float support on AVR) and ANSI color codes via `colorCode()` for visual distinction of alert states in the serial terminal.

The report format includes:

- Sensor name and current raw value
- Threshold bounds (low–high)
- Current alert state with color coding
- Debounce counter value
- Alert summary (OK / HIGH! / LOW!)

3.3 Layered Design (Hardware-Software)

This project reuses the LED and Serial drivers from previous laboratory works [1, 2]. The distance sensor driver is new for this lab.

3.3.1 Distance Sensor Driver

The Distance Sensor driver follows the same layered architecture. Figure 8 shows the layer decomposition.

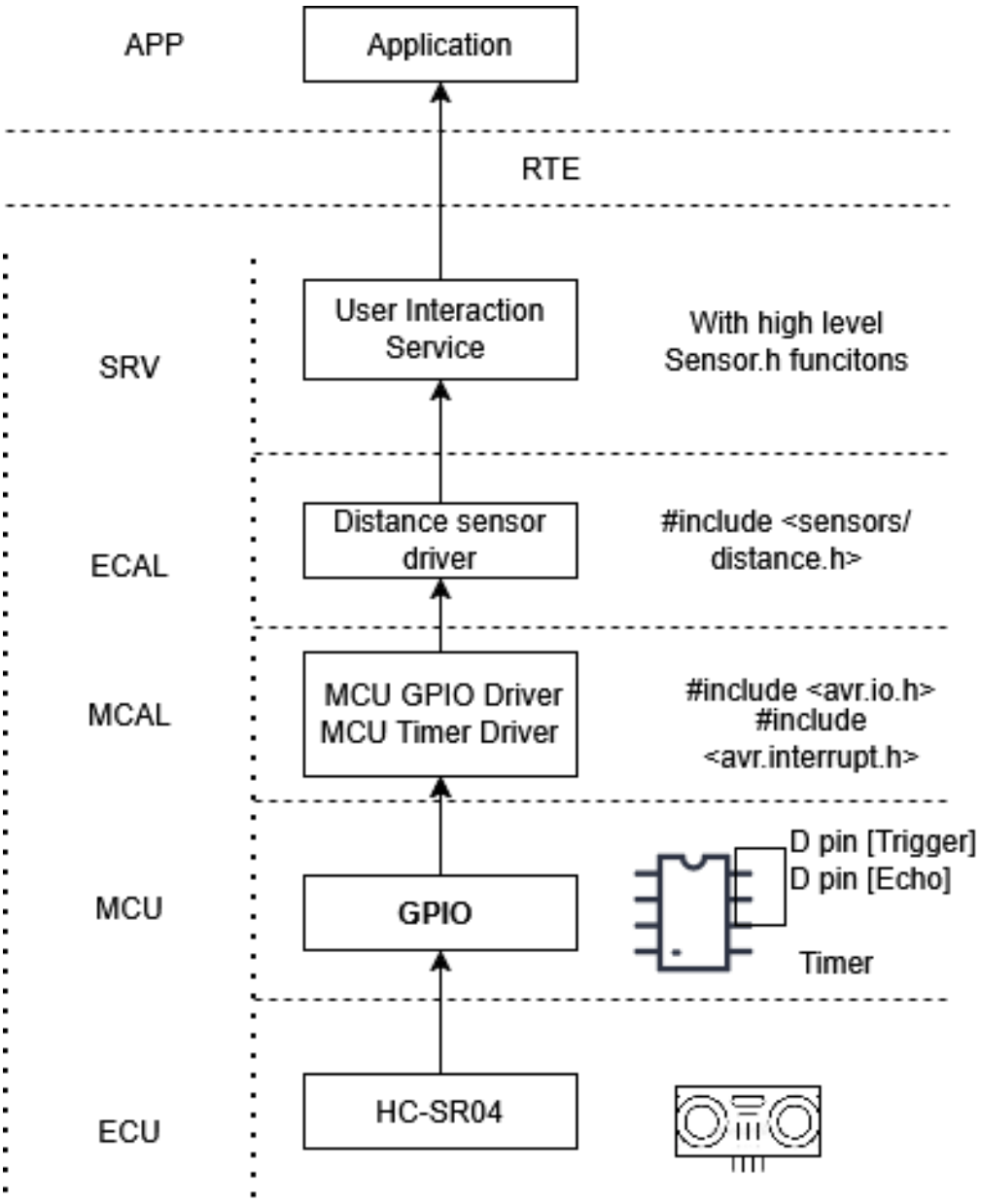

Figure 8: Distance Sensor layering (MCAL/ECAL/APP)

The layers are the following:

1. **APP** – Application Layer: The `GenericReadingSensorTask` calls `sensor.readRaw()` to obtain distance measurements.
2. **RTE** – Runtime Environment: Routes calls between the application task and the sensor driver.
3. **SRV** – Service Layer: Contains the Sensor Acquisition Service, handles timing and data storage in `SensorData`.
4. **ECAL** – ECU Abstraction Layer: Contains the `DistanceSensor` class (`distance.h/.cpp`), provides `readRaw()` wrapping the GPIO trigger/echo sequence.
5. **MCAL** – Microcontroller Abstraction Layer: Arduino `digitalWrite()`, `pulseIn()`, `pinMode()` abstracting ATmega328P GPIO and timer peripherals.
6. **MCU** – Microcontroller Unit: Physical GPIO pins D9 (TRIG, output) and D10 (ECHO, input) on the ATmega328P.
7. **ECU** – Electronic Control Unit: The HC-SR04 ultrasonic sensor module hardware.

3.4 Electrical Schematics

The electrical circuit of the system is represented in Figure 9. It consists of an HC-SR04 ultrasonic distance sensor connected to the Arduino Nano via TRIG (D9) and ECHO (D10) pins, two LEDs (Red on D12, Green on D13) with current-limiting resistors, and the UART interface for serial communication.

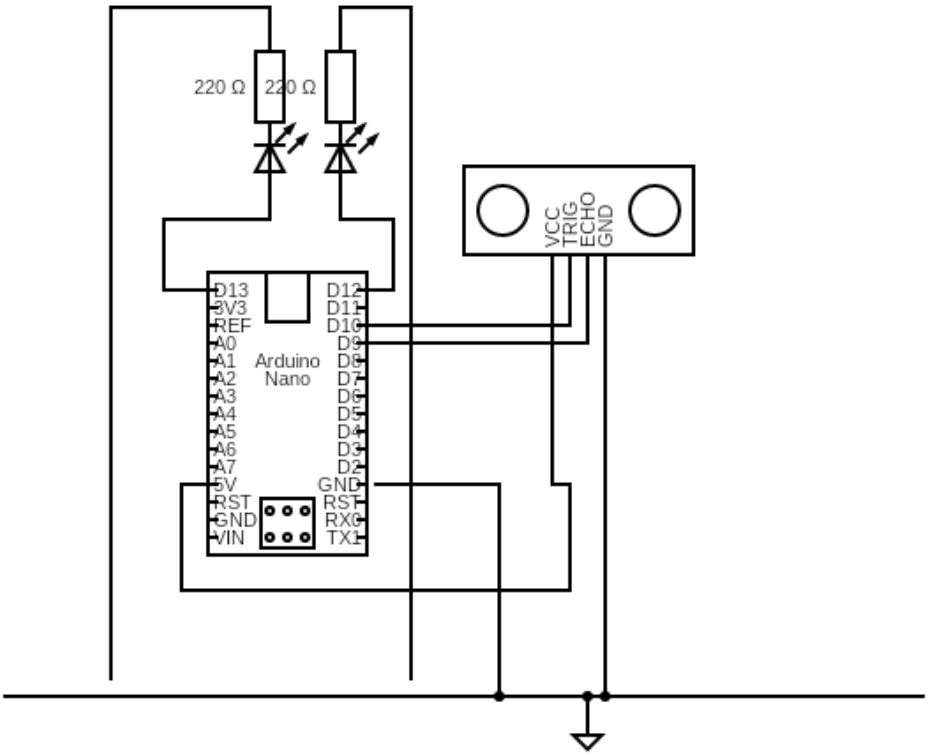

Figure 9: Electrical Schematic of the Signal Acquisition System

3.5 Simulation

The simulation of the system is performed using Wokwi or physical testing with the Arduino Nano board.

When an object is placed closer than 23 cm (below the low threshold), the system transitions through ALERT_PENDING_LOW (debouncing for 3 consecutive readings), then to ALERT_ACTIVE_LOW, triggering the Red LED and displaying the "LOW!" alert in the serial report.

When the object is moved beyond 27 cm (above the high threshold), the system enters ALERT_ACTIVE_HIGH after debouncing, again activating the Red LED and reporting "HIGH!" in the serial output.

When the object is within the normal range (23–27 cm), the Green LED is on and the report shows "OK" status.

3.6 Challenges

During this laboratory work, several challenges were addressed:

1. **HC-SR04 timing in FreeRTOS**: The `pulseIn()` function is blocking and can take up to 30ms for maximum range measurements. Since this runs within a FreeR-

TOS task, it temporarily blocks the acquisition task but does not affect other tasks due to preemptive scheduling.

2. **Volatile shared data**: The `SensorData` struct fields accessed by multiple tasks (`rawValue`, `alertState`, `debounceCount`, `alertActive`) are declared `volatile` to prevent compiler optimizations that could cause stale reads. For atomic compound operations, a mutex would be needed, but the current design relies on single-word atomic reads/writes on the 8-bit AVR.

3. **Float formatting on AVR**: The standard `printf` with `%f` format is not linked by default on AVR to save flash space. The `splitFloat()` utility function splits a float into integer and fractional parts for formatting with `%ld.%02lu`, avoiding the need for the large floating-point printf library.

4. **Stack sizing**: Each FreeRTOS task requires a separate stack. The Acquisition and Threshold tasks use 96 words (192 bytes) each, while the Report task uses 150 words (300 bytes) due to the `printf` formatting overhead. Careful sizing is critical on the ATmega328P's 2KB SRAM.

4 Conclusion

This laboratory work successfully demonstrated a complete signal acquisition and binary conditioning pipeline on a resource-constrained embedded system. By combining an HC-SR04 ultrasonic distance sensor with a FreeRTOS-based multitask architecture, we implemented:

- **Periodic Acquisition**: The sensor is read every 50ms by a dedicated high-priority task, ensuring consistent sampling regardless of other system activities.

- **Binary Conditioning with Hysteresis**: The 7-state FSM with configurable thresholds (25 ± 2 cm) effectively prevents chattering at boundary values, a critical requirement for reliable alert systems.

- **Software Debouncing**: The counter-based debounce mechanism (3 consecutive readings required) filters transient spikes, confirming alerts only after sustained threshold violations ($3 \times 100\text{ms} = 300\text{ms}$ confirmation delay).

- **Structured Reporting**: The 500ms periodic report provides complete system visibility with raw values, threshold bounds, state machine state, and color-coded alerts.

Key design decisions that proved effective:

- The CRTP pattern eliminated virtual function overhead while maintaining polymorphism, impacting the final binary size on AVR.
- The template-based `GenericReadingSensorTask<SensorType>` allows the same acquisition framework to be reused with any sensor implementing `readRaw()`.
- Separating acquisition, conditioning, and reporting into independent tasks with different priorities and intervals provides clean separation of concerns and allows each stage to operate at its optimal frequency.

The system demonstrates that even on a highly constrained microcontroller (ATmega328P, 2KB RAM, 32KB Flash), sophisticated signal conditioning algorithms can be implemented effectively using an RTOS-based architecture.

References

[1] Lab 1.1 report with Led module implementation and explanation. https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_1/FAF-231_SI_LAB1-1_Timur-Cravtov.pdf

[2] Lab 1.2 report with Serial module implementation and explanation. https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_2/FAF-231_SI_LAB1-2_Timur-Cravtov.pdf

[3] FreeRTOS, *FreeRTOS Real Time Kernel (RTOS) – API Reference*, https://www.freertos.org/a00106.html

[4] Richard Barry, *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*, https://www.freertos.org/Documentation/RTOS_book.html

[5] feilipu, *Arduino FreeRTOS Library*, https://github.com/feilipu/Arduino_FreeRTOS_Library

[6] HC-SR04 Ultrasonic Sensor Datasheet, https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[7] SEI Indrumar Lab 4, Lab 3.1: Senzori – Achizitie si conditionare semnal.

[8] Shared Code Library - Timur Cravtov https://github.com/TimurCravtov/EmbeddedSystemsLabs

A Source Code

Besides this appendix, the full source code is available in the GitHub repository [8].

`main.cpp`

```
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include <Arduino.h>
#include <stdio.h>

#include <led/led.h>
#include <serialio/serialio.h>
#include <sensors/distance.h>

#include "GenericReadingSensorTask.h"
#include "ThresholdTask.h"
#include "ReportTask.h"

// —— Sensor configuration ——
const SensorConfig distanceConfig = {
    50,      // readingIntervalMs   (20–100 ms)
    100,     // thresholdIntervalMs
    25.0,    // thresholdCenter     (cm)
    2.0,     // hysteresis          (+– cm)
    3,       // debounceRequired
    "DIST"   // name
};

const uint16_t REPORT_RECURRENCE_MS = 500;

// —— Hardware pins ——
const uint8_t TRIGGER_PIN    = 9;
const uint8_t ECHO_PIN       = 10;
const uint8_t RED_LED_PIN    = 12;
const uint8_t GREEN_LED_PIN = 13;

// —— Objects ——
Led redLed(RED_LED_PIN);
Led greenLed(GREEN_LED_PIN);

```

```
DistanceSensor distanceSensor(TRIGGER_PIN, ECHO_PIN);
GenericReadingSensorTask<DistanceSensor> distanceTask(distanceSensor, distanceConfig);

// —— Shared sensor list (add more entries for a second sensor) ——
SensorData* sensorList[] = { distanceTask.getDataPtr() };
const uint8_t SENSOR_COUNT = sizeof(sensorList) / sizeof(sensorList[0]);

// —— Task 2: One ThresholdTask instance per sensor ——
ThresholdTask distanceThresholdTask(distanceTask.getDataPtr());

// —— Task 3: Report params ——
void onAlertUpdate(bool anyAlert) {
    if (anyAlert) {
        redLed.on();
        greenLed.off();
    } else {
        redLed.off();
        greenLed.on();
    }
}

ReportTaskParams reportParams = {
    sensorList,
    SENSOR_COUNT,
    REPORT_RECURRENCE_MS,
    onAlertUpdate
};

void setup() {
    Serial.begin(9600);
    redirectSerialToStdio(true, true, true);

    printf("[Setup] Init...\n");

    redLed.init();
```

```
    greenLed.init();
    greenLed.on();
    distanceSensor.init();

    xTaskCreate(
        GenericReadingSensorTask<DistanceSensor>::taskEntryPoint,
        "Acquire",
        96,
        &distanceTask,
        3,
        NULL
    );

    xTaskCreate(
        ThresholdTask::taskEntryPoint,
        "Thresh",
        96,
        &distanceThresholdTask,
        2,
        NULL
    );

    xTaskCreate(
        ReportTask::run,
        "Report",
        150,
        &reportParams,
        1,
        NULL
    );

    vTaskStartScheduler();
}

void loop() { }
```

Listing 1: main.cpp (lab3_1)

`GenericTask.h`

```cpp
#pragma once
#include <stdint.h>

template <typename Derived>
class GenericTask {
public:
    void run(void* parameters = nullptr) {
        static_cast<Derived*>(this)->_run(parameters);
    }
};
```

Listing 2: GenericTask.h

`GenericReadingSensorTask.h`

```cpp
#pragma once

#include "GenericTask.h"
#include <Arduino_FreeRTOS.h>

enum AlertState : uint8_t {
    ALERT_NORMAL,           // within threshold band
    ALERT_PENDING_HIGH,     // debouncing to enter HIGH alert
    ALERT_ACTIVE_HIGH,      // above high threshold, alert confirmed
    ALERT_PENDING_EXIT_HIGH, // debouncing to exit HIGH alert
    ALERT_PENDING_LOW,      // debouncing to enter LOW alert
    ALERT_ACTIVE_LOW,       // below low threshold, alert confirmed
    ALERT_PENDING_EXIT_LOW     // debouncing to exit LOW alert
};

struct SensorConfig {
    uint16_t readingIntervalMs;  // acquisition recurrence (20-100 ms)
    uint16_t thresholdIntervalMs; // conditioning recurrence
    float thresholdCenter;       // center threshold value
    float hysteresis;            // +- band around center
    uint8_t debounceRequired;    // consecutive readings to confirm state change
```

```
    const char* name;              // sensor label for reporting
};

struct SensorData {
    volatile float rawValue;
    float thresholdHigh;
    float thresholdLow;
    volatile AlertState alertState;
    volatile uint8_t debounceCount;
    volatile bool alertActive;  // true if ALERT_ACTIVE_HIGH or
        ALERT_ACTIVE_LOW
    const char* name;
    uint16_t thresholdIntervalMs;
    uint8_t debounceRequired;
};

// Task 1: Acquisition just reads raw sensor data periodically
// SensorType must provide: float readRaw()
template <typename SensorType>
class GenericReadingSensorTask : public GenericTask<
    GenericReadingSensorTask<SensorType>> {
private:
    SensorType& sensor;
    SensorConfig config;
    SensorData data;

public:
    GenericReadingSensorTask(SensorType& sensor, const
        SensorConfig& cfg)
        : sensor(sensor), config(cfg)
    {
        data.rawValue            = 0;
        data.thresholdHigh       = cfg.thresholdCenter + cfg.
            hysteresis;
        data.thresholdLow        = cfg.thresholdCenter - cfg.
            hysteresis;
        data.alertState          = ALERT_NORMAL;
        data.debounceCount       = 0;
```

```
        data.alertActive          = false;
        data.name                 = cfg.name;
        data.thresholdIntervalMs  = cfg.thresholdIntervalMs;
        data.debounceRequired     = cfg.debounceRequired;
    }

    SensorData* getDataPtr() { return &data; }

    static void taskEntryPoint(void* parameters) {
        auto* instance = static_cast<GenericReadingSensorTask*>(
            parameters);
        instance->_run(parameters);
    }

    void _run(void* parameters) {
        while (true) {
            data.rawValue = sensor.readRaw();
            vTaskDelay(pdMS_TO_TICKS(config.readingIntervalMs));
        }
    }
};
```

Listing 3: GenericReadingSensorTask.h

`ThresholdTask.h`

```
#pragma once

#include "GenericReadingSensorTask.h"
#include "GenericTask.h"
#include <Arduino_FreeRTOS.h>

class ThresholdTask : public GenericTask<ThresholdTask> {
private:
    SensorData* data;
    void processThreshold();

public:
    ThresholdTask(SensorData* sensorData);
    static void taskEntryPoint(void* parameters);
    void _run(void* parameters);
```

```
};
```

Listing 4: ThresholdTask.h

ThresholdTask.cpp

```
#include "ThresholdTask.h"

ThresholdTask::ThresholdTask(SensorData* sensorData) : data(
    sensorData) {}

void ThresholdTask::taskEntryPoint(void* parameters) {
    auto* instance = static_cast<ThresholdTask*>(parameters);
    instance->_run(parameters);
}

void ThresholdTask::_run(void* parameters) {
    while (true) {
        processThreshold();
        vTaskDelay(pdMS_TO_TICKS(data->thresholdIntervalMs));
    }
}

void ThresholdTask::processThreshold() {
    float value = data->rawValue;

    switch (data->alertState) {
    case ALERT_NORMAL:
        // Check if going above high threshold
        if (value >= data->thresholdHigh) {
            data->alertState = ALERT_PENDING_HIGH;
            data->debounceCount = 1;
        }
        // Check if going below low threshold
        else if (value <= data->thresholdLow) {
            data->alertState = ALERT_PENDING_LOW;
            data->debounceCount = 1;
        }
        break;

    case ALERT_PENDING_HIGH:
```

```
        if (value >= data->thresholdHigh) {
            if (++data->debounceCount >= data->debounceRequired) {
                data->alertState = ALERT_ACTIVE_HIGH;
                data->alertActive = true;
            }
        } else if (value <= data->thresholdLow) {
            // Switched to going low
            data->alertState = ALERT_PENDING_LOW;
            data->debounceCount = 1;
        } else {
            // Back to normal range
            data->alertState = ALERT_NORMAL;
            data->debounceCount = 0;
        }
        break;

    case ALERT_ACTIVE_HIGH:
        // Start exit when value drops back below the high threshold
        if (value < data->thresholdHigh) {
            data->alertState = ALERT_PENDING_EXIT_HIGH;
            data->debounceCount = 1;
        }
        break;

    case ALERT_PENDING_EXIT_HIGH:
        if (value < data->thresholdHigh) {
            if (++data->debounceCount >= data->debounceRequired) {
                data->alertState = ALERT_NORMAL;
                data->alertActive = false;
            }
        } else {
            // Value went back above high threshold - restore alert
            data->alertState = ALERT_ACTIVE_HIGH;
            data->debounceCount = 0;
```

```
        }
        break;

    case ALERT_PENDING_LOW:
        if (value <= data->thresholdLow) {
            if (++data->debounceCount >= data->debounceRequired) {
                data->alertState = ALERT_ACTIVE_LOW;
                data->alertActive = true;
            }
        } else if (value >= data->thresholdHigh) {
            // Switched to going high
            data->alertState = ALERT_PENDING_HIGH;
            data->debounceCount = 1;
        } else {
            // Back to normal range
            data->alertState = ALERT_NORMAL;
            data->debounceCount = 0;
        }
        break;

    case ALERT_ACTIVE_LOW:
        // Start exit when value rises back above the low threshold
        if (value > data->thresholdLow) {
            data->alertState = ALERT_PENDING_EXIT_LOW;
            data->debounceCount = 1;
        }
        break;

    case ALERT_PENDING_EXIT_LOW:
        if (value > data->thresholdLow) {
            if (++data->debounceCount >= data->debounceRequired) {
                data->alertState = ALERT_NORMAL;
                data->alertActive = false;
            }
        } else {
```

```
            // Value went back below low threshold - restore alert
            data->alertState = ALERT_ACTIVE_LOW;
            data->debounceCount = 0;
        }
        break;
    }
}
```

Listing 5: ThresholdTask.cpp

ReportTask.h

```
#pragma once

#include "GenericReadingSensorTask.h"
#include <serialio/serialio.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>

struct ReportTaskParams {
    SensorData** sensors;
    uint8_t sensorCount;
    uint16_t reportIntervalMs;
    void (*onAlertUpdate)(bool anyAlert);
};

const char* alertStateStr(AlertState s);
ConsoleColor alertStateColor(AlertState s);

namespace ReportTask {
    void run(void* parameters);
}
```

Listing 6: ReportTask.h

ReportTask.cpp

```
#include "ReportTask.h"

const char* alertStateStr(AlertState s) {
    switch (s) {
```

```
        case ALERT_NORMAL:              return "NORMAL";
        case ALERT_PENDING_HIGH:        return "PEND_HI";
        case ALERT_ACTIVE_HIGH:         return "HIGH!";
        case ALERT_PENDING_EXIT_HIGH:   return "EXIT_HI";
        case ALERT_PENDING_LOW:         return "PEND_LO";
        case ALERT_ACTIVE_LOW:          return "LOW!";
        case ALERT_PENDING_EXIT_LOW:    return "EXIT_LO";
        default:                        return "?";
    }
}

ConsoleColor alertStateColor(AlertState s) {
    switch (s) {
        case ALERT_NORMAL:              return ConsoleColor::GREEN;
        case ALERT_PENDING_HIGH:        return ConsoleColor::YELLOW;
        case ALERT_ACTIVE_HIGH:         return ConsoleColor::RED;
        case ALERT_PENDING_EXIT_HIGH:   return ConsoleColor::YELLOW;
        case ALERT_PENDING_LOW:         return ConsoleColor::YELLOW;
        case ALERT_ACTIVE_LOW:          return ConsoleColor::MAGENTA;  // or CYAN for low alert
        case ALERT_PENDING_EXIT_LOW:    return ConsoleColor::YELLOW;
        default:                        return ConsoleColor::WHITE;
    }
}

void ReportTask::run(void* parameters) {
    auto* p = static_cast<ReportTaskParams*>(parameters);

    while (true) {
        bool anyAlert = false;

        printf("===== Report ===== \n");
```

```
        for (uint8_t i = 0; i < p->sensorCount; i++) {
            SensorData* s = p->sensors[i];

            int32_t rawW;   uint32_t rawF;
            int32_t hiW;    uint32_t hiF;
            int32_t loW;    uint32_t loF;
            splitFloat(s->rawValue,        2, rawW, rawF);
            splitFloat(s->thresholdHigh, 1, hiW,  hiF);
            splitFloat(s->thresholdLow,  1, loW,  loF);

            printf("[%s] Val:%ld.%02lu\n", s->name, rawW, rawF);
            printf(" Thr:%ld.%lu-%ld.%lu\n", loW, loF, hiW, hiF);

            char* clr = colorCode(alertStateColor(s->alertState));
            char* rst = colorCode(ConsoleColor::WHITE);
            printf(" St:%s%s%s Db:%u\n", clr, alertStateStr(s->alertState), rst, s->debounceCount);

            // Show alert direction
            const char* alertText;
            char* alertClr;
            if (s->alertState == ALERT_ACTIVE_HIGH) {
                alertText = "HIGH!";
                alertClr = colorCode(ConsoleColor::BRIGHT_RED);
            } else if (s->alertState == ALERT_ACTIVE_LOW) {
                alertText = "LOW!";
                alertClr = colorCode(ConsoleColor::BRIGHT_MAGENTA);
            } else {
                alertText = "OK";
                alertClr = colorCode(ConsoleColor::BRIGHT_GREEN);
            }
            printf(" Alert:%s%s%s\n", alertClr, alertText, rst);

            if (s->alertActive) anyAlert = true;
```

```
        }
        printf("==================\n");

        if (p->onAlertUpdate) {
            p->onAlertUpdate(anyAlert);
        }

        vTaskDelay(pdMS_TO_TICKS(p->reportIntervalMs));
    }
}
```

Listing 7: ReportTask.cpp

`distance.h (shared library)`

```
#pragma once

#include <Arduino.h>
#include <stdint.h>


template <typename Derived>
class Sensor {
public:
    void init() {
        static_cast<Derived*>(this)->_init();
    }

    float readRaw() {
        return static_cast<Derived*>(this)->_readRaw();
    }
};

class DistanceSensor: public Sensor<DistanceSensor> {
public:
    DistanceSensor(uint8_t triggerPin, uint8_t echoPin);
    float _readRaw();
    void _init();
private:
    uint8_t triggerPin;
    uint8_t echoPin;
```

```
};
```

Listing 8: distance.h

`distance.cpp (shared library)`

```
#include "sensors/distance.h"

DistanceSensor::DistanceSensor(uint8_t triggerPin, uint8_t
    echoPin) : triggerPin(triggerPin), echoPin(echoPin) { }

void DistanceSensor::_init() {
    pinMode(this->triggerPin, OUTPUT);
    pinMode(this->echoPin, INPUT);
}



float DistanceSensor::_readRaw() {
    digitalWrite(this->triggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(this->triggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(this->triggerPin, LOW);

    long duration = pulseIn(this->echoPin, HIGH);
    float distance = duration * 0.034 / 2; // Speed of sound is
        343 m/s, which is 0.034 cm/us
    return distance;
}
```

Listing 9: distance.cpp